

	Permissions affected by umask are calculated from the base (default) permissions (666 for files and 777 for directories), not from the current or existing permissions on the OS.	
S09	On directories, execute (x) is mandatory to access, delete, or enter files—even if you have write permission.	True
S10	Environment variables can contain configuration settings and paths needed by programs.	True

PART 2: LAB. EXERCISES – LINUX SHELL COMMANDS (15 POINTS)

EXERCISE 1 (3 = 4*0.75 POINTS) : Linux Command Errors: Identification and Troubleshooting

For each command below, identify the error it produces, explain why the command fails, and suggest a correct way to fix it.

Commands		Error / Reason for Failure / Corrected Command
01	<code>cd /home/*D</code>	Error : bash: cd: too many arguments Explanation (Reason for Failure): *D is a wildcard that matches multiple directories in /home (e.g., /home/Desktop, /home/Downloads). - cd can only take one directory at a time. - Since the pattern expands to more than one match, Bash reports “too many arguments”. Fix (Corrected Command): Specify a single directory or quote/choose one explicitly: <code>cd /home/Desktop</code>
02	<code>touch desktop/ff</code>	Error : touch: cannot touch /home/user/desktop/ff: No such file or directory Explanation (Reason for Failure): - desktop/ (lowercase) does not exist. - Touch cannot create a file in a non-existent directory. - On many systems, the desktop folder is capitalized: ~/Desktop. Fix (Corrected Command): <code>touch ~/Desktop/ff</code> Ensure the directory exists before creating a file.
03	<code>cat ~/.bash_profile</code>	Error : cat: /home/user/.bash_profile: No such file or directory Explanation (Reason for Failure): - The file <code>.bash_profile</code> does not exist in the user's home directory. - On many modern Linux distributions, Bash uses <code>~/.profile</code> or <code>~/.bashrc</code> instead of <code>.bash_profile</code>. <code>cat</code> can only display existing files, it fails when the file is missing. Fix (Corrected Command): Check which shell file exists (e.g., <code>.bashrc</code> , <code>.profile</code>) or create <code>.bash_profile</code> .
04	<code>chmod 755 /etc/shadow</code>	Error : chmod: changing permissions of '/etc/shadow': Operation not permitted Explanation (Reason for Failure): <code>/etc/shadow</code> is a critical system file containing password hashes. - Only the root user is allowed to modify its permissions. - A regular user lacks the required privileges, so the kernel denies the operation. Fix (Corrected Command): - Use administrative privileges : <code>sudo chmod 755 /etc/shadow</code> - Best practice – safer : <code>sudo chmod 640 /etc/shadow</code>

EXERCISE 2 (2.5 = 10*0.25 POINTS) : Understanding Command Behaviour: Explanation and Output

Briefly describe the result of the execution of each command (or give an explanation of it)

Command		Output / Explanation
01	<code>ls /etc grep 'conf' wc -l</code>	Count the number of files in <code>/etc</code> that contain the word "conf" in their name.
02	<code>ps aux grep bash > processes.txt</code>	Save the list of running processes containing the word "bash" to the file <code>processes.txt</code> (redirects the output).
03	<code>find . -type d wc -l</code>	Count the number of directories and subdirectories in the current folder
04	<code>ls /home /non_existent_directory &> output.txt</code>	Redirects both standard output (stdout) and standard error (stderr) results of <code>ls</code> to the file <code>output.txt</code>
05	<code>echo "Important message will" > /dev/null</code>	Nothing is displayed on the screen (the message is permanently discarded) Redirecting output to <code>/dev/null</code> is commonly used to suppress command output .
06	<code>grep -v 'error' log.txt sed 's/failure/success/g'</code>	Filter out lines containing "error" then replaces "failure" with "success" in the remaining lines.

07	<code>usermod -G dev_team nscsStudent</code>	Adds the user "nscsStudent" to the "dev_team" group, making them a member of this group. It replaces the entire list of secondary groups of the user nscsStudent with only the group dev_team. – All other secondary groups are removed. – The primary group is NOT deleted (it remains unchanged).
08	<code>ln -s /path/to/original /path/to/symlink</code>	Creates a symbolic link (symlink) named /path/to/symlink that points to the original file or directory at /path/to/original.
09	<code>find /home/user -name "*.txt" -exec cp {} /backup/ \;</code>	Searches for all .txt files in the /home/user directory (and its subdirectories), then copies each found file to the /backup/ directory. The -exec option runs the cp command for each file found.
10	<code>sudo chown -R user1:usergroup /var/www/*</code>	Changes the ownership of all files and directories under /var/www/ to user1 as the owner and usergroup as the group. The -R flag applies the change recursively.

EXERCISE 3 (7.5 POINTS) : (User, Group and Permission Management in Linux) - Analyse the following scenario and answer questions.

Scenario (Shared Linux Server) :

You are the system administrator of a multi-user Linux server. Several users collaborate using shared directories with controlled access.

1. Group memberships

```
admin@server:~$ groups u1 u2 u3 u4 root
u1   : u1 g1
u2   : u2 g1 g2
u3   : u3 g2
u4   : u4
root : root
```

2. Shared directories

```
admin@server:~$ ls -ld /data /data/dirA /data/dirB
drwxr-xr-x  5 root root 4096 janv. 10 09:00 /data
drwxrwsr-x  3 u1  g1  4096 janv. 18 14:30 /data/dirA
drwxrwxrwt  4 root g2  4096 janv. 18 16:10 /data/dirB
```

3. Directory contents

```
admin@server:~$ ls -al /data/dirA /data/dirB
/data/dirA
-rw-rw----  1 u1 g1 1200 janv. 18 10:00 f1.txt
-r--r-----  1 u2 g1  500 janv. 18 11:20 f2.csv
-rwxr-x---  1 u1 g1  300 janv. 18 13:10 run.sh
/data/dirB
-rw-rw----  1 u3 g2  800 janv. 18 09:00 f3.pdf
-rw-r--r--  1 u2 g2  400 janv. 18 15:40 f4.txt
```

Questions:

Reminder :

✓ **x** = enter (cd) ✓ **r** = list contents ✓ **w** = create/delete (with x)

- **Read file** → **r** on the file + **x** on the directory
- **Modify file** → **w** on the file + **x** on the directory
- **Execute file** → **x** on the file + **x** on the directory
- **Delete file** → depends on **directory permissions**, not file permissions
- **Sticky bit (+t)** → only the **file owner, directory owner, or root** may delete

Q1. Directory access (2 pts) : For each user, indicate whether they can **enter (cd)** the directories /data/dirA and /data/dirB. Justify your answers.

User	cd dirA	cd dirB	Justification
u1	YES	YES	Owner of dirA; others have x on dirB
u2	YES	YES	Member of g1 and g2
u3	YES	YES	Others have x on dirA; group g2 on dirB
u4	YES	YES	Others have execute permission on both

Q2. File access permissions (1.5 pts) : Answer **YES** or **NO** and justify briefly.

1. Can u2 read /data/dirA/f1.txt?	YES u2 ∈ g1 Group permission: rw- → read allowed
2. Can u2 modify /data/dirA/f2.csv?	NO u2 is the owner Owner permission: r-- (no write)

3. Can u3 read /data/dirB/f3.pdf?	YES u3 is the owner Owner permission: rw- → read allowed
4. Can u1 execute /data/dirA/run.sh?	YES u1 is the owner Owner permission: rwx → execute allowed
5. Can u2 delete /data/dirB/f3.pdf?	NO (sticky bit prevents deletion by non-owner) – Sticky bit set on /data/dirB – u2 is not the file owner (u3), directory owner (root), nor root
6. Can u4 read /data/dirB/f4.txt?	YES – u4 is others – Others permission: r-- → read allowed

Q3. Special permissions (1.5 pt)

1. Identify the special permission set on /data/dirA.	/data/dirA → SGID set (drwxrwsr-x) The s in the group execute position indicates SetGID on a directory. (0.25 pt)
2. Explain its effect on new files created in this directory.	Effect of SetGID on a directory : (0.50 pt) – Any new file or subdirectory created in /data/dirA will inherit the group g1 , regardless of the creator's primary group. – This ensures consistent group ownership and smooth collaboration among users of group g1. Typical use: shared project or team directories.
3. Identify the special permission set on /data/dirB.	/data/dirB → Sticky bit set (drwxrwxrwt) The t in the others execute position indicates the sticky bit . (0.25 pt)
4. Explain why this permission is important in a shared directory.	Importance of the sticky bit : (0.50 pt) – Even though the directory is world-writable , users cannot delete or rename files they do not own . – Only the file owner , the directory owner , or root may delete files. This prevents accidental or malicious deletion of other users' files in shared directories.

Q4. Linux administration commands (1.25 pt) : Write the exact command(s) to :

1. Add user u4 to group g1.	sudo usermod -aG g1 u4 -aG → append user to the group (without removing existing groups) (0.25 pt)
2. Ensure all new files created in /data/dirA belong to group g1.	chmod g+s /data/dirA chmod g+s → enables SetGID , forcing group inheritance (0.25 pt)
3. Remove execute permission for "others" from /data/dirA/run.sh.	chmod o-x /data/dirA/run.sh Removes the execute bit (x) for others (0.25 pt)
4. List all files in /data/dirB writable by group members.	ls -l /data/dirB grep '^..w' Explanation: – ^ → start of permission string – ..w → write permission in the group position – Lists files where the group has write access (Alternative, more precise using find): find /data/dirB -type f -perm -g=w -ls (0.50 pt)

Q5. Conceptual question (1.25 pt): Explain the difference between **write permission on a file** and **write permission on a directory**, with one concrete example.

• Write permission on a file (w) : (0.25 pt)

- Write permission on a file allows a user to modify the contents of the file
- It does not allow deleting the file & renaming the file
- **Example** : -rw-r--r-- u1 g1 report.txt
User u1 can edit report.txt
User u1 cannot delete report.txt unless they have **w** permission on the directory

• Write permission on a directory (w) : (0.25 pt)

- Write permission on a directory allows a user to: **Create new files; Delete files; Rename files**
- Only effective if combined with **execute (x)** on the directory.
- **Example** : drwxrwx--- u1 g1 shared/
Members of g1 can: Create files in shared/ ; Delete or rename files in shared/
Even if they **cannot write inside the files themselves**

• **Concrete combined example :** (0.75 pt)

→ User can delete a file without write permission on the file itself if directory is writable (unless sticky bit prevents it)

• **Concrete combined example :**

- d rwx rwx --- u1 g1 shared/
- r --- r --- r --- u2 g1 shared/data.txt
- A group member **cannot modify** data.txt (file has no **w**)
- But **can delete** data.txt (directory has **w + x**)

Bonus (+0.5 pt): Why is it insecure to assign 777 permissions to /data/dirB instead of using groups and the sticky bit ?

• Using 777 → anyone can modify/delete files, insecure

With 777: d rwx rwx rwx /data/dirB

- **Any user** can delete, rename, or replace **any file**, even if they do not own it.
- This enables: Accidental deletions ; Malicious sabotage ; Data loss

Without the **sticky bit**, file ownership is meaningless in a shared directory.

• Using SGID + sticky bit → controlled collaboration, preserves ownership, more secure

Sticky bit + groups provide secure collaboration

Current secure setup: d rwx rwx rwt root g2 /data/dirB

- **Group g2** → controlled write access
- **Sticky bit (+t)** → only owners can delete their files
- **Others** → limited to safe access

Same usability, **much stronger security**.

❑ **EXERCISE 4 (2 = 8*0.25 POINTS) : (QSM / QCS) - Marks all correct statements.**

Q1	Which numeric permission ensures that only the file owner can read and write, while group and others may only read the file ?					
	chmod 777 file.txt	chmod 664 file.txt	chmod 644 file.txt	☒	chmod 600 file.txt	
Q2	Which symbolic-mode command revokes execute permission from others only, without touching owner or group permissions ?					
	chmod a-x script.sh	chmod o-x script.sh	☒	chmod go-x script.sh	chmod u-x script.sh	
Q3	Which chmod configuration correctly encodes a permission schema where the owner has full operational authority, the group has invocation-only capability, and others may merely inspect without modification ?					
	chmod 751 app.sh	☒	chmod 754 app.sh	chmod 715 app.sh	chmod 571 app.sh	
Q4	Which command applies a permission model ensuring that every file created within a directory automatically inherits the directory's group, regardless of user ownership ?					
	chmod +t shared_dir/	chmod g+s shared_dir/	☒	chmod u+s shared_dir/	chmod 777 shared_dir/	
Q5	A script must be executable for all users, modifiable only by its owner, and the directory must prevent anyone except a file's actual owner from deleting or renaming files. Which pair of commands satisfies this security model ?					
	chmod 777 deploy.sh ; chmod +t /var/scripts	chmod 700 deploy.sh ; chmod g+s /var/scripts	chmod 755 deploy.sh ; chmod +t /var/scripts	☒	chmod 755 deploy.sh ; chmod 777 /var/scripts	
Q6	The current access permissions of the file : Proj/repA/script.sh are 764. We want to allow execution only for the owner and remove execution permission from group and others , without modifying any other permissions.					
	chmod 744 Proj/repA/script.sh	chmod u+x,go-x Proj/repA/script.sh	chmod a-x Proj/repA/script.sh	chmod go-x Proj/repA/script.sh	☒	
Q7	Which of the following statements correctly distinguishes Wildcards from Regular Expressions ?					
	Regular expressions are more expressive than wildcards	☒	Wildcards support complex pattern matching such as grouping and repetition	Regular expressions are interpreted by the shell before command execution	Wildcards are commonly used for filename matching	☒
Q8	A PTS (pseudo-terminal slave) is created when:					
	THE computer boots	A disk is mounted	THE kernel crashes	A user opens a remote session via SSH	☒	